

Importing the dependencies

```
import numpy as np
import pandas as pd
import difflib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

Data Collection and Pre-Processing

```
# loading the data from the csv file to pandas dataframe
movie_data = pd.read_csv(r"/content/movies.csv")
```

```
pd.set_option("display.max_columns", None)
```

```
# printing the first 5 data of the dataframe
movie_data.head(3)
```

	index	budget	genres	homepage	id	keywords	original_language	original_title	overview	popularity
0	0	237000000	Action Adventure Fantasy Science Fiction	http://www.avatarmovie.com/	19995	culture clash future space war space colony so...	en	Avatar	In the 22nd century, a paraplegic Marine is di...	150
1	1	300000000	Adventure Fantasy Action	http://disney.go.com/disneypictures/pirates/	285	ocean drug abuse exotic island east india trad...	en	Pirates of the Caribbean: At World's End	Captain Barbossa, long believed to be dead, ha...	139
2	2	245000000	Action Adventure Crime	http://www.sonypictures.com/movies/spectre/	206647	spy based on novel secret agent sequel mi6	en	Spectre	A cryptic message from Bond's past sends him o...	107

```
# number of rows and columns in the data frame
movie_data.shape
```

```
(4803, 24)
```

```
# selecting the relevant features for recommendation
```

```
selected_features = ["genres", "keywords", "overview", "cast", "director"]
print(selected_features)
```

```
['genres', 'keywords', 'overview', 'cast', 'director']
```

```
# cheak the null valuse
```

```
movie_data[selected_features].isnull().sum()
```

```

0
genres      28
keywords    412
overview     3
cast        43
director     30

dtype: int64

```

```
movie_data[selected_features].info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4803 entries, 0 to 4802
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   genres      4775 non-null   object
1   keywords    4391 non-null   object
2   overview    4800 non-null   object
3   cast        4760 non-null   object
4   director    4773 non-null   object
dtypes: object(5)
memory usage: 187.7+ KB

```

```
# replacing the null valuse with null string
```

```
for feature in selected_features:
```

```
    movie_data[feature] = movie_data[feature].fillna('')
```

```
# combining all the 5 selected features
```

```
combined_features = movie_data["genres"]+" "+movie_data["keywords"]+" "+movie_data["overview"]+" "+movie_data["cast"]+" "+movie_data["direct
```

```
print(combined_features)
```

```

0      Action Adventure Fantasy Science Fiction cultu...
1      Adventure Fantasy Action ocean drug abuse exot...
2      Action Adventure Crime spy based on novel secr...
3      Action Crime Drama Thriller dc comics crime fi...
4      Action Adventure Science Fiction based on nove...
...
4798    Action Crime Thriller united states\u2013mexic...
4799    Comedy Romance  A newlywed couple's honeymoon ...
4800    Comedy Drama Romance TV Movie date love at fir...
4801      When ambitious New York attorney Sam is sent...
4802    Documentary obsession camcorder crush dream gi...
Length: 4803, dtype: object

```

```
# converting the text data to feature vectors
```

```
vectorizer = TfidfVectorizer()
```

```
feature_vectors = vectorizer.fit_transform(combined_features)
```

```
print(feature_vectors)
```

```

<Compressed Sparse Row sparse matrix of dtype 'float64'
with 286496 stored elements and shape (4803, 30088)>
Coords      Values
(0, 530)     0.06395123328321288
(0, 668)     0.07378222553110117
(0, 9576)    0.09113797346979598
(0, 23607)   0.08500860956041037
(0, 9842)    0.0853037546374338
(0, 6477)    0.1603349430563257
(0, 5183)    0.16672644228889755
(0, 10596)   0.12441728045310145
(0, 25016)   0.2645437924299466
(0, 28995)   0.09351539472350907
(0, 5492)    0.18530884510145168
(0, 24837)   0.14386455634547538
(0, 13234)   0.04036788240761354
(0, 26690)   0.0614122388578492

```

```
(0, 225)      0.21092490336654032
(0, 4680)     0.134346535189137
(0, 19781)    0.20603709208377594
(0, 16738)    0.15386969208429505
(0, 13776)    0.04821324579997663
(0, 7676)     0.19234250055398622
(0, 26968)    0.03407105275718965
(0, 17952)    0.15953462271750882
(0, 19716)    0.20190450652804282
(0, 19228)    0.05414279518324664
(0, 28138)    0.16672644228889755
:
(4802, 21040) 0.09305509349460295
(4802, 8996)  0.07763651274741376
(4802, 23730) 0.08659395281859657
(4802, 6020)  0.10683894324202699
(4802, 14075) 0.09850091415465094
(4802, 6761)  0.09305509349460295
(4802, 3795)  0.11014770426456842
(4802, 8100)  0.2817499856700427
(4802, 2443)  0.2888984101538764
(4802, 22716) 0.0830941415840822
(4802, 9908)  0.10328312093536345
(4802, 10539) 0.10786737320000014
(4802, 11458) 0.10328312093536345
(4802, 1782)  0.09579355030138391
(4802, 6907)  0.10896678382598157
(4802, 1768)  0.1160074477713004
(4802, 23426) 0.11432851387600652
(4802, 7810)  0.07370276442009777
(4802, 8243)  0.11142319550666369
(4802, 6418)  0.20352868096964474
(4802, 9730)  0.11142319550666369
(4802, 11311) 0.11280973342546545
(4802, 4197)  0.11788433618267007
(4802, 235)   0.12246858844730676
(4802, 12344) 0.559900488255825
```

Cosine Similarity

```
# getting the similarty scores using cosine similarity
```

```
similarity = cosine_similarity(feature_vectors)
```

```
print(similarity)
```

```
[[1.          0.03405403 0.03366926 ... 0.0198114  0.0312866  0.0070467 ]
 [0.03405403 1.          0.03105138 ... 0.03489247 0.02038452 0.01381278]
 [0.03366926 0.03105138 1.          ... 0.02802907 0.05202542 0.01180564]
 ...
 [0.0198114  0.03489247 0.02802907 ... 1.          0.03743513 0.03653328]
 [0.0312866  0.02038452 0.05202542 ... 0.03743513 1.          0.03078044]
 [0.0070467  0.01381278 0.01180564 ... 0.03653328 0.03078044 1.          ]]
```

```
similarity.shape
```

```
(4803, 4803)
```

```
# getting the movie name from the user
```

```
user_movie_name = input("Enter your favorite movie name: ")
```

```
Enter your favorite movie name: iron man
```

```
# creating a list with all the movie names given in the dataset
```

```
list_of_all_titles = movie_data['title'].tolist()
print(list_of_all_titles)
```

```
['Avatar', 'Pirates of the Caribbean: At World's End', 'Spectre', 'The Dark Knight Rises', 'John Carter', 'Spider-Man 3', 'Tangled', 'Av
```

```
# finding the close match the movie name given by the user
```

```
find_close_match = difflib.get_close_matches(user_movie_name, list_of_all_titles)
print(find_close_match)
```

➡ ['Iron Man', 'Iron Man 3', 'Iron Man 2']

```
close_match = find_close_match[0]
print(close_match)
```

→ Iron Man

```
# finding the index of the movie with title
```

```
index_of_the_movie = movie_data[movie_data.title == close_match]['index'].values[0]
print(index_of_the_movie)
```

68

```
# getting a list of similar movies
```

```
similarity_score = list(enumerate(similarity[index_of_the_movie]))
print(similarity_score)
```

```
[((0, np.float64(0.05615057997016769)), (1, np.float64(0.03401732994623017)), (2, np.float64(0.01206485953736716)), (3, np.float64(0.0071
```

```
len(similarity_score)
```

→ 4803

```
# sorting the movies based on their similarity score
sorted_similar_movies = sorted(similarity_score, key = lambda x:x[1], reverse = True)
print(sorted_similar_movies)
```

```
[68, np.float64(0.9999999999999998)), (79, np.float64(0.330288351682709)), (31, np.float64(0.2798397406063428)), (7, np.float64(0.18866
```

```
# print the name of similar movies based on the index
```

```
print("Movies suggested for you : \n")
```

$$i = 1$$

```
for movie in sorted_similar_movies:
    index = movie[0]
    title_from_index = movie_data[movie_data.index == index]["title"].values[0]
    if (i<10):
        print(i, ".", title_from_index)
        i+=1
```

➡ Movies suggested for you :

```
1 . Iron Man
2 . Iron Man 2
3 . Iron Man 3
4 . Avengers: Age of Ultron
5 . The Avengers
6 . X-Men
7 . Captain America: Civil War
8 . The Helix... Loaded
9 . Made
```

Movie Recommendation Sytem

```
user_movie_name = input("Enter your favorite movie name: ")
```

```
list_of_all_titles = movie_data['title'].tolist()
```

```
find_close_match = difflib.get_close_matches(user_movie_name,list_of_all_titles)
```

```
close match = find close match[0]
```

```
index of the movie = movie data[movie data.title == close match]['index'].values[0]
```

```
similarity_score = list(enumerate(similarity[index_of_the_movie]))
```

```
sorted_similar_movies = sorted(similarity_score,key = lambda x:x[1],reverse = True)
```

```
print("Movies suggested for you : \n")

i = 1

for movie in sorted_similar_movies:
    index = movie[0]
    title_from_index = movie_data[movie_data.index == index]["title"].values[0]
    if (i<11):
        print(i,".",title_from_index)
        i+=1
```

↗ Enter your favorite movie name: Avatar
Movies suggested for you :

- 1 . Avatar
- 2 . Lifeforce
- 3 . Gattaca
- 4 . Star Trek Beyond
- 5 . Moonraker
- 6 . Guardians of the Galaxy
- 7 . Aliens
- 8 . Lockout
- 9 . Gravity
- 10 . Alien

Start coding or [generate](#) with AI.